

CS102: Operating Systems Concepts — Page 1: Processes, Threads, and Concurrency

1. The Process Model and Process Control Block

A process is an active program in execution. The operating system maintains metadata for each process in a Process Control Block (PCB). The PCB contains: Process Identification (PID), Process State, Program Counter (PC), CPU Registers, CPU Scheduling Information (priority), Memory-Management Information (page tables), and I/O Status Information.

Process Lifecycle States:

- New: Process is being created.
- Ready: Process is loaded in memory and waiting for CPU allocation.
- Running: Instructions are being executed by the CPU core.
- Waiting/Blocked: Process is waiting for an I/O completion or event.
- Terminated: Process has completed execution.

Context Switch: The mechanism of saving the state of the active process in its PCB and restoring the state of another scheduled process. Context switches incur pure system overhead.

2. Threads and Multithreading

A thread is the smallest unit of CPU utilization within a process. Multiple threads of the same process share the code section, data section, and open files (OS resources). However, each thread retains its own private Thread ID, Program Counter, Register set, and Stack.

Benefits of multithreading: Responsiveness in interactive applications, resource sharing, economy (creating threads is faster than processes), and scalable utilization of multi-core processors.

CS102: Operating Systems Concepts — Page 2: CPU Scheduling and Synchronization

1. CPU Scheduling Algorithms

- First-Come, First-Served (FCFS): Non-preemptive. Simple FIFO queue, but suffers from the Convoy Effect where short processes wait behind long CPU bursts.
- Shortest Job First (SJF) & Shortest Remaining Time First (SRTF): Schedules the process with minimal CPU burst duration. Provably optimal in minimizing average waiting time, but susceptible to starvation of long jobs.
- Round Robin (RR): Preemptive scheduling where each ready process receives a discrete time slice (time quantum q). If quantum is excessively large, RR degenerates to FCFS; if quantum is too small, excessive context-switch overhead degrades performance.
- Multilevel Feedback Queue (MLFQ): Multiple priority queues with adaptive demotion for CPU-intensive processes and promotion for I/O-bound processes.

2. The Critical Section Problem and Synchronization

A race condition occurs when multiple threads concurrently access and mutate shared data, and the outcome depends on execution timing. The Critical Section must satisfy three criteria: Mutual Exclusion, Progress, and Bounded Waiting.

Primitives:

- Mutex Locks: Binary flag providing mutual exclusion via `acquire()` and `release()`.
- Semaphores: Integer variable S accessed through `wait()` (P) and `signal()` (V). A counting semaphore controls access to a finite set of resources.

CS102: Operating Systems Concepts — Page 3: Deadlock Principles and Management

1. The Four Coffman Deadlock Conditions

A deadlock situation in an operating system occurs if and only if all four Coffman conditions hold simultaneously:

1. Mutual Exclusion: At least one resource must be held in a non-shareable mode.
2. Hold and Wait: A process must hold at least one resource while waiting to acquire additional resources held by other processes.
3. No Preemption: Resources cannot be forcibly taken from a process holding them; they must be released voluntarily.
4. Circular Wait: A closed chain of processes exists such that each process holds at least one resource needed by the next process in the cycle.

2. Deadlock Handling Strategies

- Deadlock Prevention: Eliminates at least one of the four Coffman conditions. For example, enforcing a total resource ordering eliminates Circular Wait.
- Deadlock Avoidance: The OS inspects resource allocation requests dynamically using the Banker's Algorithm. The system determines if granting a request leaves the system in a 'Safe State' (a sequence of process completions exists where all demands can be met).
- Deadlock Detection and Recovery: Employs a Wait-For Graph (WFG) to detect cycles. Recovery is accomplished via process termination or selective resource preemption with rollback.

CS102: Operating Systems Concepts — Page 4: Virtual Memory, Paging, and Page Replacement

1. Paging Architecture and Address Translation

Virtual memory decouples logical addresses viewed by user programs from physical RAM addresses. Physical memory is segmented into fixed-size frames, and logical memory is divided into equal-sized pages (standard 4KB).

The Memory Management Unit (MMU) translates a logical address (Page Number p , Offset d) to physical address (Frame Number f , Offset d) via the Page Table.

Translation Lookaside Buffer (TLB): A high-speed associative hardware cache holding recent page-to-frame translations. Effective Memory Access Time (EMAT) = $\text{HitRate} * (\text{TLB_access} + \text{RAM_access}) + (1 - \text{HitRate}) * (\text{TLB_access} + 2 * \text{RAM_access})$.

2. Page Faults and Page Replacement Algorithms

When a process accesses a page not present in physical memory (valid-invalid bit set to invalid), a Page Fault interrupt traps to the OS kernel. The OS locates the page on backing store (swap disk), finds a free frame, reads the page into memory, updates the page table, and restarts the faulting instruction.

Algorithms for victim page selection:

- FIFO: Replaces the oldest page. Suffers from Belady's Anomaly (increasing frames can increase page faults).
- Optimal (OPT / Belady's Min): Replaces page that will not be referenced for longest future duration. Serves as a theoretical benchmark.
- Least Recently Used (LRU): Replaces page that has not been referenced for the longest period. Implemented via hardware timestamp registers or doubly linked stacks.